Stack Pointer Package Demo

Stack Pointer is a library for visualizing the execution of imperative computer programs, particularly showing effects on the call stack: stack frames and local variables.

Overview

This package helps you:

- Visualize program execution step-by-step
- Show call stack frames
- Display local variables in each frame
- Trace function calls and returns

Example: C Program Visualization

Consider this simple C program:

```c
int main() {
  int x = foo();
  return 0;
}

int foo() {
  return 0;
}
```

Typst Representation

```
#import "@preview/stack-pointer:0.1.0": *

#let steps = execute({
  let foo() = func("foo", 6, l => {
    l(0)
    l(1); retval(0)
  })
  let main() = func("main", 1, l => {
    l(0)
    l(1)
    let (x, ..rest) = foo(); rest
    l(1, push("x", x))
    l(2)
  })
  main(); l(none)
})
```

Execution Steps Visualization

The `steps` variable contains an array where each element corresponds to one line of code execution.

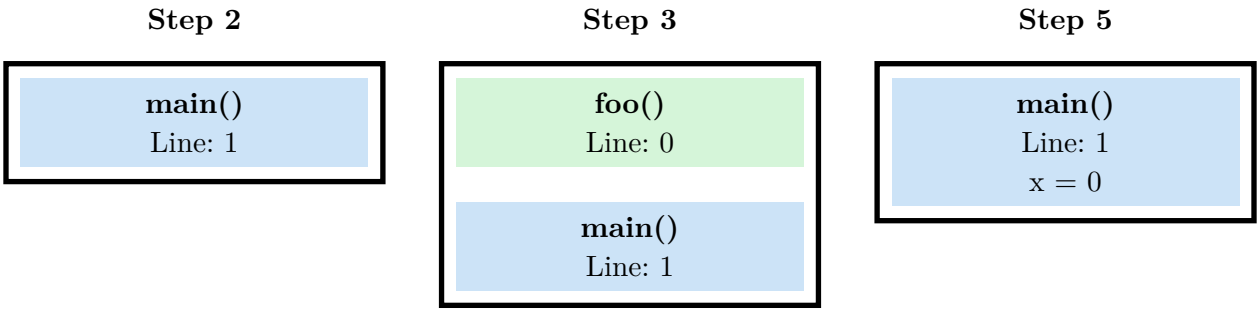
Step	Function	State
1	main	Line 0 - Enter main()
2	main	Line 1 - Before foo() call
3	foo	Line 0 - Enter foo()
4	foo	Line 1 - Return 0
5	main	Line 1 - x = 0 (from foo)
6	main	Line 2 - Return 0
7	-	Program end

Key Functions

Function	Description
`execute(body)`	Execute program and collect execution steps
`func(name, line, body)`	Define a function with name, starting line, and body
`l(line)`	Mark current line number in execution
`l(line, push(var, val))`	Mark line and push variable to stack frame
`retval(value)`	Return a value from the current function
`push(name, value)`	Add a local variable to current stack frame

Call Stack Visualization

At each step, you can visualize the call stack:



Use Cases

Teaching

- Explain function calls
- Show stack behavior
- Demonstrate recursion

Presentations

- Step through code
- Animate execution
- Works with Polylux

Integration with Polylux

Stack Pointer is designed to work with Polylux for creating animated slide presentations:

```
#import "@preview/polylux:0.3.1": *
#import "@preview/stack-pointer:0.1.0": *

#let steps = execute({ ... })

#for step in steps {
  #slide[
    // Render current execution state
    // Show stack frames
    // Highlight current line
  ]
}
```

Features Summary

- **Step-by-step execution** - Track each line of code
- **Call stack visualization** - See function stack frames
- **Variable tracking** - Monitor local variables per frame
- **Return value tracing** - Track function return values
- **Presentation integration** - Works with Polylux slides

Package: stack-pointer v0.1.0
Source: https://typst.app/universe/package/stack-pointer
Best for: CS education, teaching, presentations
Note: Requires Polylux 0.3.1 compatibility